

```

registrar int i,j;
pas[0][0]=1;
pas[1][0]=pas[1][1]=1;
para(i=2;i<=n;i++)
{
    pas[i][0]=1;
    para(j=1;j<i;j++)
        {
            paso[i][j]= paso[i-1][j-1]+pas[i-1][j];
        }
    pas[i][j]=1;
}
}

principal(void)
{
    pascales(10);
    entero n,m;
    mientras(scanf("%d%d",&n,&m)!=EOF) {

        printf("%lu",pas[n][m]); }

    devolver 0;
}

```

Combinación con objetos repetidos:

Si en una palabra de s caracteres de longitud, un carácter se repite l veces, entonces el número de arreglos es:

$$\frac{s!}{l!}$$

Si hay más de un carácter repetido, podemos escribir:

$$\frac{s!}{l_1! * l_2! * \dots * l_n!}$$

dónde,

l_1 es el número de repeticiones del primer carácter repetido. l_2

es para el segundo carácter repetido.

etcétera...

pero recuerda que calcular tanto $n!$ como $l_1! * l_2! * l_3! \dots * l_n!$ puede provocar un desbordamiento. Así que uso

$$\frac{s!}{1! \cdot 2! \cdot \dots \cdot n!}$$

$$= \frac{1 \cdot 2 \cdot 3 \cdot \dots \cdot s}{1 \cdot 2 \cdot \dots \cdot m \cdot 1 \cdot 1 \cdot 2 \cdot 3 \cdot \dots \cdot l \cdot 2 \cdot \dots \cdot 1 \cdot 2 \cdot \dots \cdot n}$$

$$= \frac{1}{1} \cdot \frac{2}{2} \cdot \dots \cdot \frac{m}{1} \cdot \frac{m+1}{1} \cdot \frac{m+2}{2} \cdot \dots \cdot \frac{m2}{l2} \cdot \dots \cdot \frac{s}{n}$$

El código de este tipo de combinación es el siguiente:

```
# incluir<stdio.h>
# incluir<math.h>
# incluir<string.h>

# define MAX 30

/* ***** */ /* Una
función de ejemplo que calcula de cuántas maneras se puede /* reorganizar una palabra con
su letra */
/* ***** */ /* double
test(char *str)
{
    entero de[MAX]={0};
    int ss[300] = {0}; int l =
    strlen(str); int i,j=0;

    doble c=1,d=1;
    para(i=0;i<l;i++)
    {
        ss[str[i]]++;
        si(ss[str[i]] > 1)
            de[j++] = ss[str[i]];
    }
    c = 1;
    para(i=2;i<=l;i++)
    {
        c*=i;

        si(j>0)
            d*= de[--j];
        if((d!=1) && !(fmod(c,d))) {
            c /= d;
            d=1;
        }
    }
    devolver c;
}

/* Una función principal de ejemplo */
```

```
principal(void)
{
    carácter palabra[MÁX];
    entero n;
    entero j=0;
    scanf("%d",&n);
    para(n>0;n--)
    {
        scanf("%s",word);
        printf("Conjunto de datos %d: %.0f",++j,test(word));
        putchar('\n');
    }
    devolver 0;
}
```

subsecuencia común más larga (LCS)

Dadas dos secuencias X e Y, decimos que una secuencia z es una **subsecuencia común** de C e Y si Z es una subsecuencia tanto de X como de Y.

subsecuencia común más larga (LCS) es simplemente el más largo "subsecuencia común" de dos secuencias.

LONGITUD LCS(X,Y)

Se reciben dos secuencias: $X < x_1, x_2, x_3, \dots, x_m >$ e $Y < y_1, y_2, y_3, \dots, y_n >$. El sistema almacena los valores $C[i,j]$ en la tabla $C[0 \dots m, 0 \dots n]$, cuyos valores se calculan en orden de filas. Además, mantiene la tabla $b[1 \dots m, 1 \dots n]$ para simplificar la construcción de una solución óptima.

```
1. m <- longitud[X]
2. n <- longitud[Y]
3. para i <- 1 a m
4.     hacer c[i,0] <- 0
5. para j <- 0 a n
6.     hacer c[0,j] <- 0
7. para i <- 1 a m
8.     para j <- 1 a n
9.         hacer si  $x_i = y_j$ 
10.            entonces  $c[i,j] <- c[i-1,j-1] + 1$ 
11.             $b[i,j] <- 1$ 
12.        else if  $c[i-1,j] >= c[i,j-1]$ 
13.            entonces  $c[i,j] <- c[i-1,j]$ 
14.             $b[i,j] <- 2$ 
15.        de lo contrario  $c[i,j] <- c[i,j-1]$ 
16. Devolver c y b
```

IMPRIMIR LCS(b,X,i,j)

```

1.sii = 0 o j = 0
2.                luego regresar
3.sib[i,j] = 1
4.                entoncesIMPRIMIR_LCS(b,X,i-1,j-1)
5.de lo contrario sib[i,j] = 2
6.                entoncesPRINT_LCS(b,X,i-1,j)
7.demás          PRINT_LCS(b,X,i,j-1)

```

Código para LCS

```

# incluir<stdio.h>
# incluir<string.h>

# define MAX 100 // tamaño de cada secuencia

char str1[MAX],str2[MAX]; // las secuencias

/* ***** */
/* Esta función imprime el LCS en la pantalla Entrada: Una          */
/* tabla generada por LCS_LNT y la longitud de las                  */
/* secuencias                                                         */
/* Salida: Ninguna                                                    */
/* ***** */

void p_lcs(int b[MAX][MAX],int i,int j) {

    si ( (i == 0) | (j == 0) ) retornar ; si( b[i][j] == 1 )

    { p_lcs(b,i-1,j-1);
      printf("%3c",str1[i-1]); }

    else if( b[i][j] == 2) p_lcs(b,i-1,j); else
        p_lcs(b,i,j-1);
}

/* ***** */
/* Esta función calcula la longitud LCS */ Entrada:                */
/* Secuencia de dos y un booleano I. Si                               */
/* Si I es FALSO (0), entonces la función no                          */
/* imprime el LCS y si es VERDADERO (1), entonces                    */
/* imprime usando el                                                  */
/* función p_lcs anterior                                             */
/* Salida: Ninguna                                                    */
/* ***** */ void
LCS_LNT(bool I)
{
    int c[MAX][MAX]={0},b[MAX][MAX]={0},l1,l2; l1 =
    strlen(str1)+1;
    l2 = strlen(str2)+1;

```

```

    registrar int i,j;
    para(i=1;i<l1;i++)
    {
        para(j=1;j<l2;j++)
        {
            if( str1[i-1] == str2[j-1] ) {
                c[i][j] = c[i-1][j-1] + 1; b[i][j] = 1;

            }
            else if(c[i-1][j] >= c[i][j-1]) {
                c[i][j] = c[i-1][j];
                b[i][j] = 2;
            }
            demás    c[i][j] = c[i][j-1];
        }
    }
    printf("%d\n",c[l1-1][l2-1]); if(I)
    p_lcs(b,l1-1,l2-1);
}

/* una función principal de ejemplo */
main(void)
{
    mientras(1)
    {
        si(!obtiene(str1)))           devolver 0;
        si(!obtiene(str2)))           devolver 0;
        LCS_LNT(1);
    }
}

```

Otro código para encontrar LCS

```

# incluir<stdio.h>
# incluir<string.h>
# define MAX 105

char str1[MAX][50],str2[MAX][50],lcs[MAX][50]; int lcswl;

/* ***** */
/*      Esta función imprime el LCS en la pantalla. Entrada: Una      */
/*      tabla generada por LCS_LNT.                                     */
/*      y la longitud de las secuencias Salida:                         */
/*      Ninguna                                                         */
/* ***** */ void
p_lcs(int b[MAX][MAX],int i,int j)
{
    si ( (i == 0) || (j == 0) ) retornar ; si( b[i][j] == 1 )

    {
        p_lcs(b,i-1,j-1);
        strcpy(lcs[lcswl++],str1[i-1]); }

    else if( b[i][j] == 2)                p_lcs(b,i-1,j);

```

```

    demás                p_lcs(b,i,j-1);
}

/* ***** */
/* Esta función calcula la longitud LCS Entrada:          */
/* Secuencia de remolque y un booleano I. Si              */
/* Si I es FALSO (0), entonces la función no              */
/* imprime el LCS y si es VERDADERO (1), entonces         */
/* imprime usando el                                     */
/* función p_lcs anterior                                 */
/* Salida: Ninguna                                         */
/* ***** */ void
LCS_LNT(int l1, int l2, bool I)
{
    int c[MAX][MAX]={0},b[MAX][MAX]={0}; registrar
    int i,j;
    para(i=1;i<l1;i++)
    { para(j=1;j<l2;j++)
      { if( !(strcmp(str1[i-1],str2[j-1])) ) {
        c[i][j] = c[i-1][j-1] + 1; b[i][j] = 1;

      }
      else if(c[i-1][j] >= c[i][j-1]) {
        c[i][j] = c[i-1][j];
        b[i][j] = 2;
      }
      demás    c[i][j] = c[i][j-1];
    }
  }
  si(yo)
  {
    lcswl = 0;
    p_lcs(b,l1-1,l2-1);
    j = c[l1-1][l2-1];
    printf("%s",lcs[0]);
    for(i = 1; i< j ;i++)                printf(" %s",lcs[i]);
    putchar("\n");
  }
}

/* Función principal de ejemplo */
main(void)
{
    carácter palabra[50];
    entero i=0,j=0,l1,l2,ln;
    mientras(scanf("%s",word)!=EOF) {

        ln = strlen(palabra);
        si(ln==1)
            si(palabra[0] == '#')
            {
                si(i==0)
                {
                    i = 1;

```

```

        l1 = j;
        j = 0;
        continuar;
    }
    demás
    {
        l2 = j;
        j = i = 0;
        prueba(l1+1,l2+1,1);
        continuar;
    }
}
si(i==0) strcpy(str1[j++],word);
    demás strcpy(str2[j++],word);
}
devolver 0;
}

```

Multiplicación de Cadenas de Matrices (MCM)

Fondo

Si dos matrices A y B cuya dimensión es (m,n) y (n,p) respectivamente, entonces la multiplicación de A y B requiere una multiplicación escalar $m \cdot n \cdot p$.

Supongamos que se le da una secuencia de n matrices $A_1, A_2, \dots, A_{\text{norte}}$. Matriz A tiene dimensión (P_{i-1}, P_i) . Tu tarea es poner entre paréntesis el producto $A_1, A_2, \dots, A_{\text{norte}}$ de tal manera que se minimice el número de productos escalares.

Como la multiplicación de matrices es asociativa, todas las operaciones con paréntesis dan como resultado el mismo producto.

ORDEN DE CADENA MATRIZ(P)

La entrada es una secuencia $P = \langle P_0, P_{A1}, P_{A2}, \dots, P_{A_{\text{norte}}} \rangle$ donde $\text{length}[P] = n+1$. El procedimiento utiliza una tabla auxiliar $m[1 \dots n, 1 \dots n]$ para almacenar los costos $m[i, j]$ y una tabla auxiliar $s[1 \dots n, 1 \dots n]$ que registra qué índice de k alcanza el costo óptimo al calcular $m[i, j]$.

1. $n \leftarrow \text{longitud}[P]-1$
2. para $i \leftarrow 1$ a n
3. hacer $m[i, i] \leftarrow 0$
4. para $l \leftarrow 2$ hasta n
5. hacer para $i \leftarrow 1$ hasta $n - l + 1$
6. hacer $j \leftarrow i + l - 1$

```

7.          m[i,j] <- INF          // INF es infinito
8.          para k <- i hasta j -1 hacer q <-
9.          m[i,k]+m[k+1,j]+ Pi-1PAGkPAGj
10.         si q < m[i,j]
11.         entonces m[i,j] <- q
12.         s[i,j] <- k
13. devolver m,s

```

IMPRIMIR(s,i,j)

El siguiente procedimiento recursivo imprime una parentetización óptima de $(A_i, A_{i+1}, \dots, A_j)$ dada la tabla s calculada por

ORDEN DE LA CADENA DE LA MATRIZ y los índices i y j. La llamada inicial PRINT(s,1,n) imprime la parentetización óptima de $(A_1, A_2, \dots, A_{\text{norte}})$.

```

1.  sii == j
2.      entonces imprimir "A"i
3.      demás  imprimir "("
4.          IMPRIMIR(s,i,s[i,j])
5.      IMPRIMIR(s,s[i,j]+1,j)
6.      imprimir ")"

```

Código:

```

# incluir<stdio.h>
# define MAX 15          /* número de matriz */
# define INF 4294967295  /* número máximo de multiplicaciones */

entero num;
/* ***** */
/* Imprime la secuencia */
/* Entrada: Una matriz bidimensional (creada */
/* por la función de orden de la matriz chan) con */
/* su punto de partida */
/* Retorno: Ninguno */
/* ***** */ void
print(unsigned long s[][MAX],int i,int j)
{
    si(i==j)
        printf("A%d",num++);
    demás
    {
        printf("(");
        imprimir(s,i,s[i][j]);
        printf(" x ");
        imprimir(s,s[i][j]+1,j);
        printf(")");
    }
}

```



```

/* ***** */
/* Descubre el orden */
/* Entrada: Una secuencia de dimensión y la */
/* número de matriz */
/* Retorno: ninguno */
/* ***** */

void matrix_chan_order(int *p,int n) {

    unsigned long m[MAX][MAX] = {0};
    unsigned long s[MAX][MAX] = {0}; unsigned
    int q;
    entero l,j,i,k;
    para(l = 2; l <= n ;l++) {

        para(i = 1 ; i <= n - l + 1 ; i++) {

            j = i + l - 1;
            m[i][j] = INF;
            para(k=i ; k < j ; k++) {

                q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j]; si(q < m[i][j])

                {
                    m[i][j] = q;
                    s[i][j] = k;
                }

            }

        }

        num = 1;
        imprimir(s,1,n);
    }

}

/* Una función principal de ejemplo */

principal(void)
{

    entero n;
    entero p[MAX]={0};
    entero i;

    mientras(scanf("%d",&n),n) {

        para(i=1;i<=n;i++)
            scanf("%d %d",&p[i-1],&p[i]);
        matrix_chan_order(p,n);
        putchar('\n');

    }

    devolver 0;
}

```

Suma de números grandes

```
# incluir<stdio.h>
# incluir<stdlib.h>
# incluir<string.h>
# incluir<math.h>
# define MAX 1000
void reverse(char *from, char *to){
    entero len = strlen(from);
    int l;
    para(l=0;l<len;l++){
        a[l]=desde[len-l-1];
        to[l]=a[l];
    }
}
void call_sum(char *first, char *sec, char *result){
    char F[MAX], S[MAX], Res[MAX]; int
    f,s,sum,extra,now;
    f=strlen(first);
    s=strlen(sec);
    invertir(primer, F);
    invertir(sec, S);
    para(ahora=0,extra=0;(ahora<f && ahora<s);ahora++){
        suma=(F[ahora]-'0') + (S[ahora]-'0') + extra;
        Res[ahora]=suma%10 +'0';
        extra = suma/10;
    }
    para(;ahora<f;ahora++){
        suma=F[ahora] + extra-'0';
        Res[ahora]=suma%10 +'0';
        extra=suma/10;
    }
    para(;ahora<s;ahora++){
        suma=F[ahora] + extra-'0';
        Res[ahora]=suma%10 +'0';
        extra=suma/10;
    }
    if(extra!=0) Res[now++]=extra+'0';
    Res[now]='\0';
    if(strlen(Res)==0) strcpy(Res,"0");
    reverse(Res,result);}

int main(){
    char fir[MAX],sec[MAX],res[MAX];
    while(scanf("%s%s",&fir,&sec)==2){
        call_sum(fir,sec,res);
        entero len=strlen(res);
        for(int i=0;i<len;i++) printf("%c",res[i]); printf("\n");
    }
    devolver 0;
}
```

Resta de números grandes

```

/*****          Resta de números grandes          *****/
# incluir<stdio.h>
# incluir<stdlib.h>
# incluir<string.h>
# incluir<math.h>
# define MAX 1000 /
*****/ void
reverse(char *from, char *to){
    entero len = strlen(from);
    int l;
    para(l=0;l<len;l++)
        a[l]=desde[len-l-1];
    to[len]='\0';
}
int call_minus(char *large, char *small, char *result){
    carácter L[MAX], S[MAX];
    int l,s,ahora,mantener,diferencia;
    l=strlen(large);
    s=strlen(small);
    signo booleano = 0;
    si(l<s){
        strcpy(resultado,large);
        strcpy(grande,pequeño);
        strcpy(small,result);
        ahora=l; l=s; s=ahora;
        signo = 1;
    }//devuelve 0;
    si(l==s){
        si(strcmp(grande,pequeño)<0){
            strcpy(resultado,large);
            strcpy(grande,pequeño);
            strcpy(small,result);
            ahora=l; l=s; s=ahora;
            signo =1;
        }//devuelve 0;
    }
    invertir(grande,L);
    invertir(pequeño,S);
    para(;s<l;s++)
        S[s]='0';
    S[s]='\0';
    para(ahora=0,mantener=0;ahora<l;ahora++){
        diff=L[ahora]-(S[ahora]+mantener);
        si(diff<0){
            mantener=1;
            resultado[ahora]=10+diferencia+'0';
        }
        demás{
            resultado[ahora]=diferencia+'0';
            mantener=0;
        }
    }
}

```

```

    }
    para(ahora=l-1;ahora>0;ahora--){
        si(resultado[ahora]!='0')
            romper;
    }
    resultado[ahora+1]='\0';
    invertir(resultado,L);
    strcpy(resultado,L);
    //devuelve 1;
    signo de retorno;
}
int main(){
    char fir[MAX],sec[MAX],res[MAX];
    while(scanf("%s%s",&fir,&sec)==2){
        si(call_minus(fir,sec,res)==1)
            printf("-");
        int len = strlen(res); para(int
        i=0;i<len;i++)
            printf("%c",res[i]);
        printf("\n");

    }
    devolver 0;
}

```

Multiplicación de números grandes

```

/*****      Multiplicación de números grandes      *****/
# incluir<stdio.h>
# incluir<stdlib.h>
# incluir<string.h>
# incluir<math.h>
# define MAX 1000 /
*****/ void
reverse(char *from, char *to ){
    entero len = strlen(from);
    int l;
    para(l=0;l<len;l++)
        a[l]=desde[len-l-1];
    to[len]='\0';
}
/*****/ void
call_mult(char *first,char *sec,char *result){
    char F[MAX],S[MAX],temp[MAX]; int
    f_len,s_len,f,s,r,t_len,hold,res; f_len=strlen(first);

    s_len=strlen(sec);
    invertir(primerio,F);
    invertir(sec,S);
    t_len=f_len+s_len;
    r=-1;

```

```

    para(f=0;f<=t_len;f++)
        temp[f]='0';
    temp[f]='\0';
    para(s=0;s<s_len;s++){
        mantener=0;
        para(f=0;f<f_len;f++){
            res=(F[f]-'0')*(S[s]-'0') + hold+(temp[f+s]-'0'); temp[f+s]=res%10+'0';

            mantener=res/10;
            si(f+s>r) r=f+s;
        }
        mientras(mantener!=0){
            res=hold+temp[f+s]-'0';
            hold=res/10;
            temp[f+s]=res%10+'0';
            si(r<f+s) r=f+s;
        }
        f++;
    }
    para(r>0 && temp[r]=='0';r--);
    temp[r+1]='\0';
    invertir(temp,resultado);
}
/*****/ int main(){

    char fir[MAX],sec[MAX],res[MAX];
    while(scanf("%s%s",&fir,&sec)==2){
        llamar_multiplicar(fir,sec,res);
        int len=strlen(res);
        for(int i=0;i<len;i++) printf("%c",res[i]); printf("\n");

    }
    devolver 0;
}

```

División de números grandes y resto

```

/*****                               División de números grandes                               *****/
# incluir<stdio.h>
# incluir<stdlib.h>
# incluir<string.h>
# incluir<math.h>
# define MAX 1000 /
*****/ int
call_div(char *number,long div,char *result){
    int len=strlen(número); int
    ahora;
    extra largo;
    char Res[MAX];
    para(ahora=0,extra=0;ahora<len;ahora++){
        extra = extra * 10 + (número[ahora] - '0');
    }
}

```

```

        Res[ahora]=extra / div +'0';
        extra%=div;
    }
    Res[ahora]='\0';
    para(ahora=0;Res[ahora]!='0';ahora++);
    strcpy(resultado, &Res[ahora]);
    si(strlen(resultado)==0)
        strcpy(resultado, "0");
    devolver extra;
}
/*****/ int main(){

    char fir[MAX],res[MAX]; long
    sec,remain;
    mientras(scanf("%s%ld",&fir,&sec)==2){
        if(sec==0) printf("Error de división por cero\n"); else{

                resto = call_div(fir,sec,res); int len  =
                strlen(res);
                for(int i=0;i<len;i++) printf("%c",res[i]);
                printf("\t%ld",remainder);
                printf("\n");

            }
        }
    devolver 0;
}

```

Raíz cuadrada de números grandes

```

/*****                               Número grande raíz cuadrada                               *****/
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<math.h>
#define MAX 1000 /
*****/ int
call_minus(char *large, char *small, char *result){
    carácter L[MAX], S[MAX];
    int l,s,ahora,mantener,diferencia;
    l=strlen(large);
    s=strlen(small);
    si(l<s)
        devolver 0;
    si(l==s){
        si(strcmp(grande, pequeño)<0)
            devolver 0;
    }
    invertir(grande,L);
    invertir(pequeño,S);
    para(s<l;s++)
        S[s]='0';
}

```

```

S[s]='\0';
para(ahora=0,mantener=0;ahora<l;ahora++){
    diff=L[ahora]-(S[ahora]+mantener);
    si(diff<0){
        mantener=1;
        resultado[ahora]=10+diferencia+'0';
    }
    demás{
        resultado[ahora]=diferencia+'0';
        mantener=0;
    }
}
para(ahora=l-1;ahora>0;ahora--){
    si(resultado[ahora]!='0')
        romper;
}
resultado[ahora+1]='\0';
invertir(resultado,L);
strcpy(resultado,L);
devolver 1;
}
/*****/ void
call_sqrt(char *number,char *result,char *extra){
    int num,start,e,mul,l,r=0,len; char
    left[MAX],after[MAX]; char
    who[5],temp[MAX],two[5];
    len=strlen(number);
    si(len%2==0){
        num=10*(numero[0]-'0') + numero[1]-'0'; inicio=2;
    }
    demás{
        num=numero[0]-'0';
        inicio=1;
    }
    mul=(int) sqrt(num);
    resultado[0]=mul+'0';
    resultado[1]='\0';
    si(num-mul*mul == 0)
        extra[0]='\0';
    demás
        sprintf(extra,"%d",num-mul*mul);
    for(;start<len;start+=2){
        e=strlen(extra);
        extra[e]=numero[inicio];
        extra[e+1]=numero[inicio+1];
        extra[e+2]='\0';
        dos[0]='2';
        dos[1]='\0';
        llamar_multiplicar(resultado,dos,izquierda);
        l=strlen(izquierda);
        para(mul=9;mul>=0;mul--){
            quien[0]=mul+'0';
            quien[1]='\0';

```

```

        strcat(izquierda,quién);
        llamar_multiplicar(izquierda,quién,después);
        si(llamar_menos(extra,después,temp)==1){
            resultado[++r]=mul+'0';
            resultado[r+1]='\0';
            strcpy(extra,temp);
            romper;
        }
    demás
        izquierda[l]='\0';
    }
    resultado[++r]='\0';
}
/*****/ int main(){

    char fir[MAX],ex[MAX],res[MAX];
    while(scanf("%s",&fir)==1){
        llamar_sqrt(fir,res,ex);
        entero len=strlen(res);
        for(int i=0;i<len;i++) printf("%c",res[i]); printf("\n");

    }
    devolver 0;
}

```

Números de Fibonacci

```

/* Esta ejecución tiene un tiempo de O(log n)
# incluir<stdio.h>
conquistar_fibonacci_largo(long n){
    largo i,h,j,k,t;
    i=h=1;
    j=k=0;
    mientras(n>0){
        si(n%2==1){
            t=j*h;
            j=i*h + j*k + t;
            i=i*k + t;
        }
        t=h*h;
        h=2*k*h + t;
        k=k*k + t;
        n=(long) n/2;}
    devolver j;}
int main(){
    largo n,res;
    mientras(scanf("%ld",&n)==1){
        res=conquer_fibonacci(n);
        printf("%ld\n",res);}
    devolver 0;
}

```


Número de Fibnacci por número entero grande

```

# incluir<iostream.h>
# incluir<string.h>
int main()
{

    char *fibo[5001]={0};
    fibo[0]="0";
    fibo[1]="1";
    int l1=strlen(fibo[0]); int
    l2=strlen(fibo[1]); int l;

    para(long i=2;i<=5000;i++)
    {
        carácter str[10000];
        si(l1>=l2)l=l1;
        de lo contrario l=l2;
        entero ca=0;
        j,k,m,p largos;
        para(j=l1-1,k=l2-1,m=0,p=0;p<l;j--,k--,m++,p++) {

            entero s1;
            if(j<0)fibo[i-2][j]='0'; s1=fibo[i-2]
            [j]-48;
            entero s2;
            if(k<0)fibo[i-1][k]='0'; s2=fibo[i-1]
            [k]-48;
            entero ans=0;
            ans+=s1+s2+ca;
            si(respuesta>9)
            {
                str[m]=(ans-10)+48;
                ca=1;
            }
            demás
            {
                str[m]=ans+48;
                ca=0;
            }
        }
        if(ca>0){str[m]=ca+48; m++;}
        str[m]='\0';
        fibo[i]=nuevo carácter[m+1];
        y=0;
        para(long x=m-1;x>=0;x--,y++)fibo[i][y]=str[x]; fibo[i][y]='\0';

        l1=strlen(fibo[i-1]);
        l2=strlen(fibo[i]);

    }
}

```

```

    entero n;

    mientras(cin>>n)
    {
        cout << "El número de Fibonacci para " << n << " es " << fibo[n] << "\n"; }

    devolver 0;
}

```

BFS/DFS (Problema del laberinto)

Aporte :

```

8 8
#.#.#.#.
.....
#.#.....
#.#.#.#..
..#...#.#
#...#.#...
...#...#
#.s#d.#.

```

```

# incluir<stdio.h>
# incluir<conio.h>
# incluir<valores.h>

# define N 100
# define MAX MAXINT

int mat[N][N], Q[N][3], cost[N][N], front = -1, rear = -1; int m, n, sc, sr, p, nr, nc, r,
c, leaf, i, j, res[10][10]; int R[4] = {0, -1, 0, 1};

int C[4] = {-1, 0, 1, 0};

void nQ(int r, int c, int p) {

    Q[++trasero][0] = r, Q[trasero][1] = c, Q[trasero][2] = p;}

void dQ(int *r, int *c, int *p) {

    *r = Q[++front][0], *c = Q[front][1], *p = front;
}

void bfs(int sr, int sc, int p) {

    nQ(sr, sc, p);
    hacer{
        dQ(&r, &c, &p);
        para(int i=0; i<4; i++) {

```

```

        nr = r + R[i], nc = c + C[i]; si(mat[nr]
[nc]==1)
        {
            si(costo[nr][nc]>costo[r][c]+1)
                costo[nr][nc]=costo[r][c]+1, nQ(nr, nc, p);
        }
    }
    } mientras (trasero!=delantero);
    hoja = p;
}
void show()
{
    para(int i=0; i<=m; i++) {
        para (int j=0; j<=n; j++) {
            si(res[i][j])
                printf("X");
            demás
                printf(" ");
        }
        printf("\n");
    }
}
void dfs(int hoja)
{
    si(Q[hoja][2]==-1)
    {
        res[Q[hoja][0]][Q[hoja][1]] = 1; retornar;
    }
    dfs(Q[hoja][2]);
    res[Q[hoja][0]][Q[hoja][1]] = 1;
}
void main()
{
    clrscr();
    carácter ch;
    freopen("maze.txt", "r", stdin);
    scanf("%d%d", &m, &n);
    obtener carácter();
    para(i=0; i<m; i++)
    {
        para(j=0; j<n; j++)
        {
            costo[i][j] = MÁX;
            scanf("%c", &ch);
            si(ch=='#')
                mat[i][j] = 0;
            else if(ch=='.')
                mat[i][j] = 1;
            else if(ch=='s')

```

```

                                mat[i][j] = 2, sc = j, sr = i;
                                demás
                                mat[i][j] = 3, res[i][j] = 1;
                                }
                                obtener carácter();
                                }
                                bfs(sr, sc, -1);
                                dfs(hoja); mostrar();
                                }

```

Suma mínima (DFS/cola)

Aporte :

```

5 6
3 4 1 2 8 6
6 1 8 2 7 4
5 9 3 9 9 5
8 4 1 3 2 6
3 7 2 1 2 3

```

Código

```

# incluir<stdio.h>
# incluir<conio.h>
# incluir<valores.h>

# define N 100
# define MAX MAXINT

int mat[N][N], M[N][N], Q[N][4], front = -1, rear = -1; int m, n, nc, nr, p, s,
finalSum=MAX, leaf, r, c, i;

void init()
{
    para(int i=0; i<N; i++)
        para(int j=0; j<N; j++)
            M[i][j] = MAX, mat[i][j] = 0;
}

void nQ(int r, int c, int p, int s) {
    Q[++trasera][0] = r;      Q[trasera][1] = c;      Q[trasera][2] = p;
    Q[trasera][3] = s;}
void dQ(int *r, int *c, int *p, int *s) {

    *r = Q[++front][0];
    *c = Q[front][1];
    *p = frente;
    *s = Q[front][3];
}

```

```
void bfs()
{
    para(r=0, c=0; r<m; r++)
        nQ(r, c, -1, mat[r][c]);

    hacer
    {
        dQ(&r, &c, &p, &s);
        si(c<n-1)
            para(i=-1; i<2; i++)
            {
                nr=(m+r+i)%m, nc=c+1;
                si(M[nr][nc] > s+mat[nr][nc])
                    nQ(nr, nc, p, s+mat[nr][nc]), M[nr][nc] = s+mat[nr][nc]; }

        else if(s<finalSum)
            suma final = s, hoja = p;
    } mientras(trasero!=delantero);
}

void dfs(int hoja)
{
    si(Q[hoja][2]==-1)
    {
        printf(" <%d, %d>", Q[leaf][0]+1, Q[leaf][1]+1); return;
    }
    dfs(Q[hoja][2]);
    printf(" <%d, %d>", Q[leaf][0]+1, Q[leaf][1]+1);
}

void main()
{
    clrscr();
    int i, j, t;
    inicializar();
    freopen("in.txt", "r", stdin);
    scanf("%d%d", &m, &n);
    para(i=0; i<m; i++)
        para(j=0; j<n; j++)
        {
            scanf("%d", &t);
            mat[i][j] = t;
        }

    bfs();

    printf("Suma final: %d\nRuta:", finalSum);

    dfs(hoja);
}
```

Floyd Warshal

Aporte

```
5 7
1 2 4
1 3 1
1 5 6
2 5 3
2 4 1
3 2 1
4 5 1
0 0
```

Código:

```
# incluir<stdio.h>
# incluir<valores.h>

# define N 100
# define INF MAXINT

entero mat[N][N], path[N][N], n, e;

void initMat()
{
    para(int i=1; i<=n; i++)
        para(int j=1; j<=n; j++)
            mat[i][j] = INF;
}

void initPath()
{
    para(int i=1; i<=n; i++)
        para(int j=1; j<=n; j++)
            si(mat[i][j]!=INF)
                ruta[i][j] = j;
            demás
                ruta[i][j] = 0;
}

void floyd_warshall()
{
    para(int k=1; k<=n; k++)
        para(int i=1; i<=n; i++)
            para(int j=1; j<=n; j++)
                si(mat[i][k]!=INF && mat[k][j]!=INF)
                    si(mat[i][k]+mat[k][j] < mat[i][j])
                        mat[i][j] = mat[i][k] + mat[k][j],
                        ruta[i][j] = ruta[i][k];
}
```

```
}  
  
void mostrarRuta(int i, int j) {  
    si(i==j)  
    {  
        printf("->%d", i);  
        devolver;  
    }  
    printf("->%d", i);  
    mostrarRuta(ruta[i][j], j);  
}  
  
void main()  
{  
    mientras(scanf("%d%d", &n, &e) && n && e)  
    {  
        initMat();  
        para (int i, j, c, k=0; k<e; k++) {  
            scanf("%d%d%d", &i, &j, &c); mat[i][j]  
            = c;  
        }  
        initPath();  
        floyd_warshall();  
        para(i=1; i<=n; i++)  
        {  
            para(j=1; j<=n; j++)  
            si(ruta[i][j])  
            {  
                printf("%d", i);  
                mostrarRuta(ruta[i][j], j);  
                imprimirf("\n");  
            }  
            printf("\n");  
        }  
    }  
}
```

Coloreado de gráficos

```
# incluir<stdio.h>  
int a[20][20],x[20],n,m; void  
next(int k)  
{  
    entero j;  
    mientras(1)
```

```

    {
        x[k]=(x[k]+1)%(m+1);
        si(x[k]==0)
            devolver;
        para(j=1;j<=n;j++)
            si(a[k][j]!=0&& x[k]==x[j])
                romper;
        si(j==n+1)
            devolver;
    }
}
void mcolor(int k)
{
    entero j;
    mientras(1)
    {
        siguiente(k);
        si(x[k]==0)
            devolver;
        si(k==n)
        {
            printf("      ");
            para(j=1;j<=n;j++)
                printf("%2d",x[j]);

        }
        demás
            mcolor(k+1);
    }
}
void main()
{
    entero i,u,v;
    printf("\n\n Ingrese cuántos colores: "); scanf("%d",&m);

    printf("\n\n Ingrese cuántos nodos (0<n<20): "); scanf("%d",&n);

    printf("\n\n Introduzca sus aristas (ej. u sp v)(presione 'e' para finalizar): \n"); for(i=1;i<=(n*n)/2;i++)
    {
        si(getchar()=='e')
            romper;
        scanf("%d%d",&u,&v);
        a[u][v]=a[v][u]=1;
    }mcolor(1); printf("\n\n");}

```

Detección de ciclo (Hamiltoniano)

```

# incluir<stdio.h>
entero a[20][20],x[20],n;
void siguiente(int k)
{
    entero j;

```



```
mientras(1)
{
    x[k]=(x[k]+1)%(n+1);
    si(x[k]==0)
        devolver;
    si((a[x[k-1]][x[k]]!=0) {
        para(j=1;j<=k-1;j++)
            si(x[k]==x[j])
                romper;
        si(j==k)
            si((k<n) || (k==n&& a[x[n]][x[1]]!=0))
                devolver;
    }
}
}
void hamilt(int k)
{
    entero j;
    mientras(1)
    {
        siguiente(k);
        si(x[k]==0)
            devolver;
        si(k==n)
        {
            printf(" ");
            para(j=1;j<=n;j++)
                printf("%2d",x[j]);
        }
        demás
            hamilt(k+1);
    }
}
void main()
{
    entero i,u,v;
    x[1]=1;
    printf("\n\n Ingrese cuántos nodos (0<n<20): "); scanf("%d",&n);

    printf("\n\n Introduzca sus aristas (ej. u sp v)(presione 'e' para finalizar): \n"); for(i=1;i<=(n*n)/2;i++)
    {
        si(getchar()=='e')
            romper;
        scanf("%d%d",&u,&v);
        a[u][v]=a[v][u]=1;
    }
    hamilt(2);
    printf("\n\n");
}
```

Encontrar el punto de articulación

```

# incluir<stdio.h>
int d[11],num=1,b[11][11],l[11],at[11],s=1; void art(int u,int
v)
{
    entero i,j=1,w,f=0;
    d[u]=num;
    l[u]=num;
    num++;
    para(i=1;b[u][i]!=0;i++) {

        w=b[u][i];
        si(d[w]==0)
        {
            arte(w,u);
            l[u]=(l[u]<l[w])?l[u]:l[w];
        }
        de lo contrario si(w!=v)
            l[u]=(l[u]<d[w])?l[u]:d[w];
        si(d[u]<=l[w])
            f=1;
    }
    si(f)
        en[s++]=u;
}
void main()
{
    int i,j,a[11][11],n,u,v,k,f=0;
    para(i=1;i<11;i++)
        para(j=1;j<11;j++)
            a[i][j]=0;
    printf("\n\n Ingrese cuántos nodos (0<n<11): "); scanf("%d",&n);

    printf("\n\n Introduzca sus aristas (ej. u sp v)(pulse 'e' para finalizar: "); for(i=1;i<=(n*n)/2;i++)
    {
        si(getchar()=='e')
            romper;
        scanf("%d%d",&u,&v);
        a[u][v]=a[v][u]=1;
    }
    para(i=1;i<=n;i++)
    {
        k=1;
        para(j=1;j<=n;j++)
            si(a[i][j])
            {
                b[i][k]=j;
                k++;
            }
    }
}

```

```
        b[i][k]=0;
    }
    para(j=1,i=1;b[1][j]!=0;j++) {

        k=b[1][j];
        si(b[k][2])
            i++;
    }
    si(j==i)
        f=1;
    arte(1,1);
    en[s]=-9;
    printf("\n\n Los puntos de articulación son: "); if(!f)

        para(i=1;en[i]!=-9;i++)
            printf("%3d",at[i]);

    si(f)
        para(i=1;en[i]!=1;i++)
            printf("%3d",at[i]);
    printf("\n\n");
}
```

APÉNDICE C

BIBLIOTECA DE PLANTILLAS ESTÁNDAR (STL)



Los lenguajes de programación para el concurso serán C, C++, Pascal y Java, así como la familia de funciones `scanf()`/`printf()`. También estarán disponibles la biblioteca de cadenas de C++ y la Biblioteca de Plantillas Estándar (STL). Esta parte del libro contiene una nota introductoria sobre STL.^[9]

Biblioteca de plantillas estándar (STL)

La biblioteca de plantillas estándar, o *STL*, es una biblioteca de C++ de clases de contenedores, algoritmos e iteradores; proporciona muchos de los algoritmos y estructuras de datos básicos de la informática. La STL es una *genérico* Se trata de una biblioteca, lo que significa que sus componentes están altamente parametrizados: casi todos los componentes de la STL son plantillas. Es importante que comprendas cómo funcionan las plantillas en C++ antes de usar la STL.

Contenedores y algoritmos

Al igual que muchas bibliotecas de clases, la STL incluye *recipiente* Clases: clases cuyo propósito es contener otros objetos. La STL incluye las clases `vector`, `list`, `deque`, `set`, `multiset`, `map`, `multimap`, `hash_set`, `hash_multiset`, `hash_map` y `hash_multimap`. Cada una de estas clases es una plantilla y puede instanciarse para contener cualquier tipo de objeto. Por ejemplo, se puede usar un `vector<int>` de forma muy similar a como se usaría un array C ordinario, con la diferencia de que `vector` elimina la necesidad de gestionar manualmente la asignación dinámica de memoria.

```
vector<int> v(3);           // Declara un vector de 3 elementos.  
v[0] = 7;  
v[1] = v[0] + 3; v[2] = v[0] +  
v[1];                     // v[0] == 7, v[1] == 10, v[2] == 17
```

El STL también incluye una gran colección de *algoritmos* que manipulan los datos almacenados en contenedores. Por ejemplo, se puede invertir el orden de los elementos de un vector utilizando el algoritmo de inversión.

```
inversa(v.begin(), v.end()); // v[0] == 17, v[1] == 10, v[2] == 7
```

Hay dos puntos importantes a tener en cuenta acerca de esta llamada a `reverse`. Primero, es una función global, no una función miembro. Segundo, toma dos argumentos en lugar de uno: opera sobre un *rango* de elementos, en lugar de en un contenedor. En este caso particular, el rango resulta ser todo el contenedor `v`.

La razón de ambos hechos es la misma: `reverse`, al igual que otros algoritmos de STL, está desacoplado de las clases de contenedores de STL.

```
double A[6] = { 1.2, 1.3, 1.4, 1.5, 1.6, 1.7 };  
invertir(A, A + 6);  
para (int i = 0; i < 6; ++i)  
    cout << "A[" << i << "] = " << A[i];
```

Este ejemplo utiliza un *rango*, al igual que en el ejemplo de invertir un vector: el primer argumento para invertir es un puntero al inicio del rango, y el segundo argumento apunta a uno

elemento más allá del final del rango. Este rango se denota $[A, A + 6)$; la notación asimétrica es un recordatorio de que los dos puntos extremos son diferentes, que el primero es el comienzo del rango y el segundo *es uno pasado* el final del rango.

Sitio web para descargar archivos STL.

<http://www.sgi.com/tech/stl/download.htm>

Archivos individuales en formato STL

algo.h	picadillo_mapa.h	numérico	stdexcept	montículo_stl.h	stl_slist.h
algbase.h	hash_sy	par.h	stl_algo.h	iterador_stl.h	pila_stl.h
algoritmo	hash_set.h	pthread_alloc	stl_algo_base.h	iterador_stl_base.h	stl_string_fwd.h
asignar.h	hashtable.h	pthread_alloc.h	stl_alloc.h	lista_stl.h	stl_tempbuf.h
conjunto de bits	montón.h	cola	stl_bvector.h	mapa_stl.h	hilos stl.h
bvector.h	iterador	soga	configuración_stl.h	stl_multimap.h	árbol_stl.h
rasgos_de_carácter.h	iterador.h	cuerda.h	construct.h	stl_multiset.h	stl_uninitialized.h
comprobación_de_conceptos.h	límites	cuerdaimpl.h	rasgos stl_fns.h	stl_numeric.h	vector_stl.h
contenedor_concepts.h	lista	secuencia_concepts.h	stl_deque.h	stl_pair.h	cadena
defalloc.h	lista.h	colocar	stl_exception.h	cola_stl.h	tempbuf.h
deque	mapa	conjunto.h	función stl Nueva Hampshire	error_de_rango_stl.sh	árbol.h
deque.h	mapa.h	lista de deseos	hash_stl_un.h	almacenamiento en bruto de stl_e_iter.h	rasgos_de_tipo.h
función.h	memoria_y	slist.h	hash_stl_mapa.h	stl_relaps.h	utilidad
funcional	multimap.h	pila	hash_stl_conjunto.h	cuerda_stl.h	valarray
mapa_hash	conjunto múltiple.h	pila.h	stl_hashtable.h	stl_set.h	vector

¿Qué compiladores son compatibles?

La STL se ha probado con los siguientes compiladores: SGI 7.1 y posteriores, o 7.0 con la opción -n32 o -64; gcc 2.8 o egcs 1.x; Microsoft 5.0 y posteriores. (Véase más abajo). Boris Fomitchev distribuye una versión para otros compiladores.

Si logras usar la STL de SGI con otro compilador, por favor, avísanos e indícanos qué modificaciones (si las hubo) tuviste que realizar. Prevemos que la mayoría de los cambios se limitarán al archivo de cabecera <stl_config.h>.

EJEMPLOS STL**Buscar**

```
const char S1[] = "Hola, mundo!"; const char
S2[] = "mundo";
const int N1 = sizeof(S1) - 1; const int N2
= sizeof(S2) - 1;

const char* p = search(S1, S1 + N1, S2, S2 + N2); printf("Se encontró la
subsecuencia \"%s\" en el carácter %d de la secuencia \"%s\".\n",
S2, p - S1, S1);
```

Cola

```
int main() {
    cola<int> Q;
    Q.push(8);
    Q.push(7);
    Q.push(6);
    Q.push(2);

    assert(Q.size() == 4);
    assert(Q.back() == 2);

    assert(Q.front() == 8); Q.pop();

    assert(Q.front() == 7); Q.pop();

    assert(Q.front() == 6); Q.pop();

    assert(Q.front() == 2);
    Q.pop();
    assert(Q.empty());}
```

Lista doblemente enlazada

```
lista<int> L;
L.empujar_atrás(0);
L.empujar_frente(1);
L.insert(++L.begin(), 2);
copy(L.begin(), L.end(), ostream_iterator<int>(cout, " ")); // Los valores que se
imprimen son 1 2 0
```

Clasificar

```
int A[] = {1, 4, 2, 8, 5, 7}; const int N = sizeof(A) /
sizeof(int); sort(A, A + N);

copy(A, A + N, ostream_iterator<int>(cout, " ")); // La salida es " 1 2
4 5 7 8".
```

Complejidad

$O(N \log(N))$ comparaciones (tanto promedio como en el peor de los casos), donde N es último - primero.

Búsqueda binaria

```
int main()
{
    int A[] = { 1, 2, 3, 3, 3, 5, 8 }; const int N = sizeof(A) /
    sizeof(int);

    para (int i = 1; i <= 10; ++i) {
        cout << "Buscando " << i << ": "
            << (binary_search(A, A + N, i) ? "presente" : "no presente") <<
    fin;
    }
}
```

La salida

Buscando 1: presente Buscando 2:
presente Buscando 3: presente
Buscando 4: no presente Buscando 5:
presente Buscando 6: no presente
Buscando 7: no presente Buscando 8:
presente Buscando 9: no presente
Buscando 10: no presente

APÉNDICE D

ordenador personal2GUÍA PARA LA ADMINISTRACIÓN DEL CONCURSO Y DEL EQUIPO



ordenador personal2es un sistema dinámico, distribuido y en tiempo real diseñado para gestionar y controlar concursos de programación. Incluye soporte para concursos multisitio, operaciones en plataformas heterogéneas que incluyen sistemas Windows y Unix mixtos en un solo concurso, y actualizaciones dinámicas en tiempo real del estado y la clasificación del concurso en todos los sitios. Aquí describimos los pasos necesarios para instalar,

Configurar y ejecutar un concurso usando una PC.2Más información sobre PC2, incluyendo cómo obtener una copia del sistema, se puede encontrar en <http://www.ecs.csus.edu/pc2>.

Software para jueces de concursos de programación - PC2

ordenador personal: Funciona mediante una arquitectura cliente-servidor. Cada sitio en un concurso ejecuta un único PC2.

servidory también ejecuta múltiples PC2 **cliente** que se comunican con el servidor del sitio. Iniciar sesión en un cliente usando uno de varios tipos diferentes de PC2 Las cuentas (Administrador, Equipo, Juez o Marcador) permiten al cliente realizar operaciones comunes del concurso asociadas al tipo de cuenta, como la configuración y el control del concurso (Administrador), el envío de programas de los concursantes (Equipo), la evaluación de las presentaciones (Juez) y el mantenimiento de la clasificación actual del concurso (Marcador).

ordenador personal: Los clientes se comunican únicamente con el servidor de su sitio, independientemente del número de sitios participantes en el concurso. En un concurso multisitio, los servidores de los sitios se comunican no solo con sus propios clientes, sino también con otros servidores de sitios para mantener un registro del estado global del concurso. Por lo tanto, se deben cumplir los siguientes requisitos de comunicación para ejecutar un concurso utilizando PC2: (1) una máquina que ejecuta una PC2 El servidor debe poder comunicarse a través de TCP/IP con cada máquina que ejecute un PC2 cliente en su sitio; y (2) en un concurso multisitio, cada máquina que ejecute una PC2 El servidor debe poder comunicarse a través de TCP/IP con las máquinas que ejecutan PC2 servidores en todos los demás sitios. En particular, no debe haber cortafuegos que prohíban estas vías de comunicación; el sistema no funcionará si se bloquea dicha comunicación. No es necesario que los equipos cliente puedan comunicarse con equipos en otras ubicaciones.

Cada PC2 El módulo (servidor o cliente) lee uno o más archivos de inicialización .ini al iniciarse; estos archivos se utilizan para configurar el módulo al arrancar. El módulo cliente también adapta su configuración cuando un usuario (equipo, juez, etc.) inicia sesión. En un PC típico En la configuración del concurso, cada equipo, juez, etc., utiliza una máquina física independiente, y cada una de estas máquinas ejecuta exactamente un módulo cliente. Es posible tener varios clientes ejecutándose en la misma máquina física, por ejemplo, con diferentes usuarios iniciando sesión en diferentes cuentas en una máquina compartida. En este caso, cada usuario (equipo, juez, etc.) ejecutará su propia máquina virtual Java (JVM) y deberá tener su propia estructura de directorios independiente, incluyendo su propia copia independiente del PC2 archivos de inicialización en su cuenta.

ordenador personal: Configuración para el administrador

Para aquellos que odian leer manuales y prefieren arriesgarse con una lista de atajos, aquí hay una *muy lacónico* Resumen de los pasos necesarios para instalar PC2 y ponerlo en marcha. Tenga en cuenta que esto se proporciona como una conveniencia para genios (o masoquistas). El resto del manual fue escrito para ayudar a todos los demás. Si tiene

* Para obtener más información sobre PC2 por favor, compruebe : <http://www.ecs.csus.edu/pc2>

problemas después de seguir esta lista, *por favor* Lea el resto del manual antes de enviarnos una solicitud de ayuda.

Instale Java (versión 1.3.1 o superior);

Instalar PC² descomprimiendo el PC² distribución al PC² directorio de instalación;

Agregue el Java *papeler* directorio y el PC² agregar el directorio de instalación a la variable PATH;

Agregue java/lib y la PC² agregar el directorio de instalación al CLASSPATH;

Modificar el *lista de sitios.ini* Archivo según sea necesario para especificar el nombre de cada servidor del sitio.

Editar el *pc2v8.ini* archivo para apuntar servidores y clientes a la IP:puerto del servidor y para especificar el nombre de servidor de sitio apropiado; coloque el archivo .ini modificado en cada máquina servidora y cliente;

Archivo pc2v8.ini de ejemplo para el sitio llamado Site

[cliente]

Dile al cliente a qué sitio pertenece

y dónde encontrar su servidor (IP y puerto) sitio=Sitio1

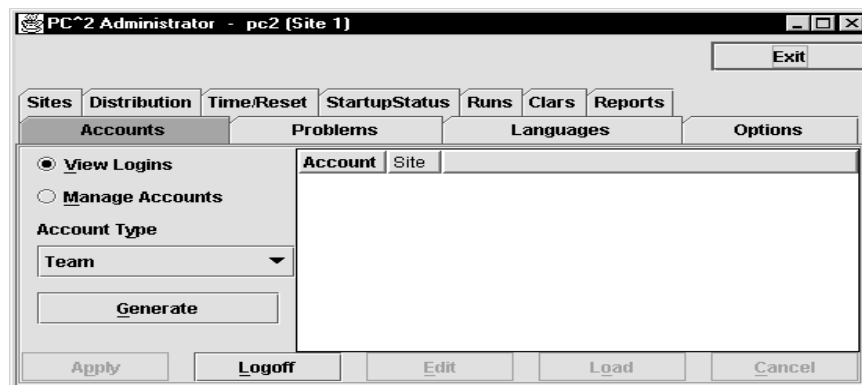
servidor=192.168.1.117:50002

[servidor]

Indica al servidor qué sitio está sirviendo: sitio=Sitio1

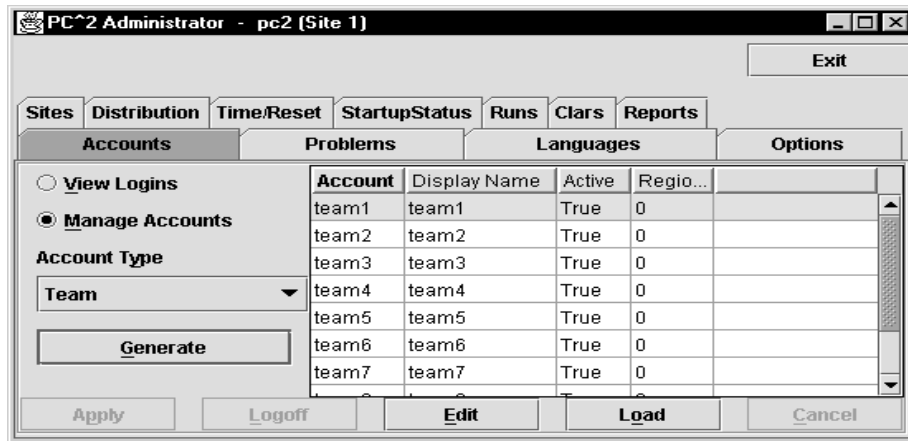
Enciende un PC² Conecte el servidor mediante el comando pc2server y responda a la pregunta que se le plantea.

Enciende un PC² El cliente administrador utiliza el comando pc2admin e inicia sesión con el nombre de usuario root y la contraseña root.

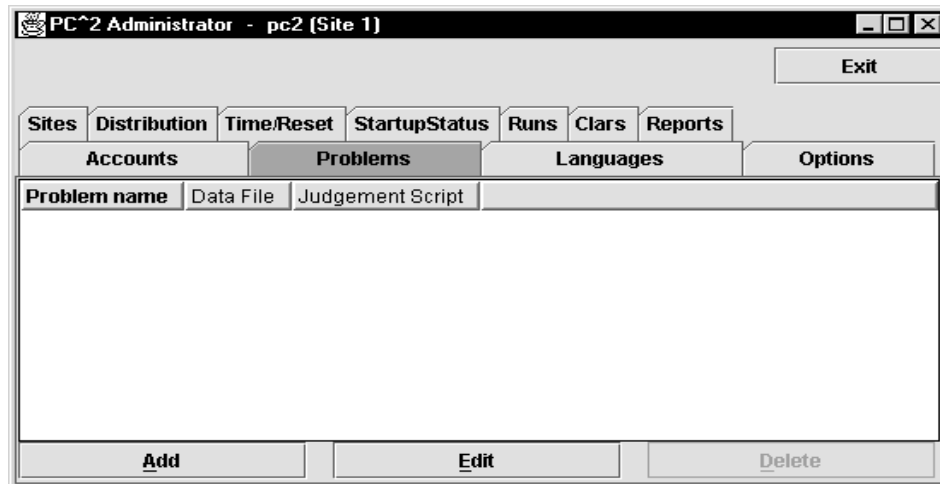


Configure al menos los siguientes elementos del concurso a través del panel de administración:

Cuentas (generar las cuentas necesarias);

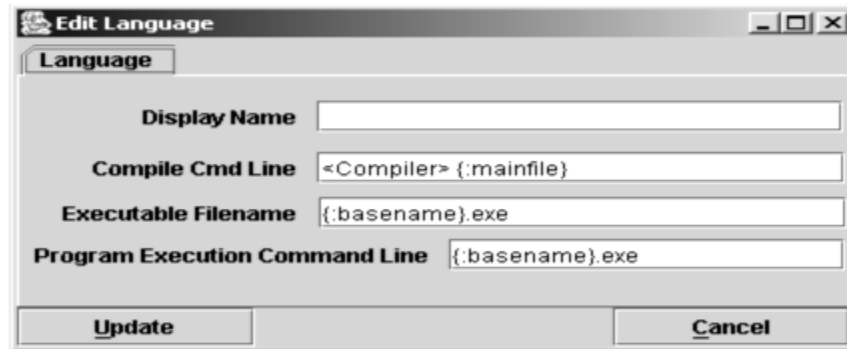


Problemas (cree uno o más problemas de concurso, especificando el archivo de datos de entrada del problema si lo hay);



Idiomas (crear idiomas para el concurso, especificando el nombre del idioma, la línea de comandos de compilación, el nombre del archivo ejecutable y la línea de comandos de ejecución).

Parámetro de lenguaje para Turbo C/C++: `tcc -Ic:\tc3\bin -Lc:\tc3\lib {mainfile}`



Pulse el botón Iniciar concurso en la pestaña Tiempo de administración/Restablecer;

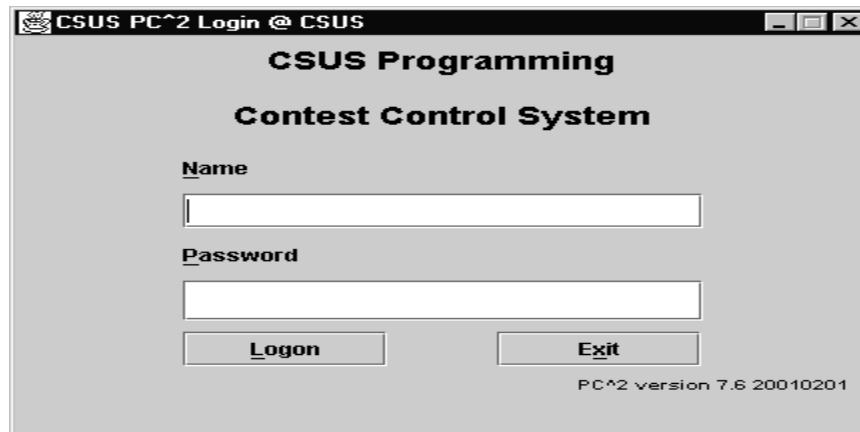
Enciende un PC2Instale el cliente en cada máquina del equipo y del juez e inicie sesión utilizando las cuentas y contraseñas creadas por el administrador.

Enciende un PC2Instale el cliente en la máquina del marcador e inicie sesión utilizando la cuenta/contraseña del marcador board1; configure los archivos HTML generados por el marcador para que sean accesibles a los navegadores de los usuarios.



ordenador personal:Guía del equipo (para los concursantes)

Esta guía tiene como objetivo familiarizarle con el proceso de envío de programas a los jueces del concurso utilizando la PC.2("PC-Squared")**PAG** programacióndo en pruebado Sistema de control. Iniciando PC2 encenderá el PC2**pantalla de inicio de sesión**, como se muestra a continuación:



CSUS PC^2 Login @ CSUS

**CSUS Programming
Contest Control System**

Name

Password

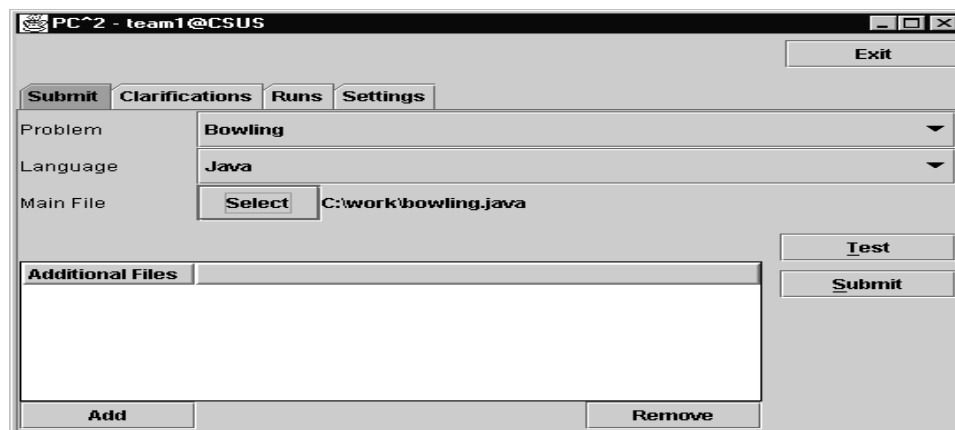
Logon **Exit**

PC^2 version 7.6 20010201

Para iniciar sesión en el PC², haga clic una vez en el **Nombre** cuadro en la pantalla de inicio de sesión, ingrese su ID de equipo asignado, presione la tecla TAB o haga clic en el **Contraseña**. En la casilla, introduzca la contraseña que se le ha asignado. Su ID de equipo tendrá el siguiente formato: **equipoxx**, donde xx es el número de equipo que se le ha asignado (por ejemplo, "equipo3" o "equipo12"). Después de ingresar el nombre de su equipo y la contraseña, haga clic en el **Iniciar sesión** botón.

Presentación de un programa a los jueces

Una vez que el sistema acepte su inicio de sesión, estará en el PC² **Menú principal** Pantalla, que se muestra a continuación. Tenga en cuenta que el ID del equipo ("team1" en este caso) y la ubicación del sitio del equipo ("CSUS" en este caso) se muestran en la barra de título.



PC^2 - team1@CSUS

Exit

Submit **Clarifications** **Runs** **Settings**

Problem **Bowling**

Language **Java**

Main File **Select** C:\work\bowling.java

Additional Files

Add **Remove**

Test **Submit**

Haciendo clic en el **ENTREGAR** La pestaña cerca de la parte superior de la pantalla muestra la **Enviar ejecución** pantalla, que se muestra arriba.

Haciendo clic en el **Problema** En este campo se mostrará una lista de los problemas del concurso; elija el problema para el que desea enviar un programa ("ejecutar") a los jueces (en el ejemplo, se ha elegido un problema llamado "Bolos").

Haciendo clic en el **Idioma** En este campo se mostrará una lista de los lenguajes de programación permitidos en el concurso; elija el lenguaje utilizado por el programa que desea enviar a los jueces (en el ejemplo, se ha elegido "Java").

Para enviar un programa a los jueces, debe especificar el nombre del archivo que contiene su **programa principal**. Haz clic en el **Seleccionar** Botón para abrir el cuadro de diálogo "Archivo", que permite localizar y seleccionar el archivo principal. El cuadro de diálogo permite navegar automáticamente a la ruta y ubicación de archivo correctas (en el ejemplo, se ha seleccionado el archivo principal del programa "C:\work\bowling.java").

Si su programa consta de más de un archivo, debe ingresar los nombres de los archivos adicionales en el **Archivos adicionales** casilla. Haga clic en la **Agregar** botón para invocar el cuadro de diálogo que le permite localizar y seleccionar sus archivos adicionales; seleccione un archivo en el cuadro y haga clic en el **Eliminar** Botón para eliminar un archivo previamente seleccionado.

Importante .*tú debe* especificar el nombre de su *programa principal* archivo en el **Archivo principal** campo, no en el **Archivos adicionales**; Seleccione solo la casilla! código fuente Archivos para su presentación a los jueces. No envíe archivos de datos ni archivos ejecutables.

ordenador personal: Desinstalar

Para desinstalar PC2Solo es necesario deshacer los pasos de instalación anteriores; es decir, Elimine el directorio \$PC2HOME y su contenido, y restaure las variables de entorno del sistema a sus valores anteriores. PC2El programa no realiza ningún cambio en ubicaciones del sistema distintas a las mencionadas anteriormente, ni durante la instalación ni durante la ejecución. En concreto, por ejemplo, no crea entradas en el registro en un entorno Windows ni copia archivos a ubicaciones fuera de los directorios de instalación en ningún entorno.

APÉNDICE E

I IMPORTANTE WSITIOS WEB/PARA ACM/ICPC P PROGRAMADORES



Hoy en día, el mundo está abierto a los estudiantes e Internet lo convierte en un mundo al alcance de nuestra mano. Existen numerosos sitios web en Internet tanto para principiantes como para programadores. Algunos ofrecen concursos y tutoriales, otros libros y otros lenguajes de programación. Las matemáticas son esenciales para la programación y existen muchos recursos matemáticos.

Sitio web en la red. Los pioneros también publican sus ideas y consejos en Internet para nosotros. En esta página encontrarás el enlace a esas páginas.^[10]

Jueces en línea y sitios de tutoriales

<http://online-judge.uva.es/> y <http://cii-judge.baylor.edu/>

El sistema de arbitraje en línea 24 horas de la Universidad de Valladolid y el archivo oficial en directo de ACM/ICPC contienen todos los problemas de los regionales de ACM.

<http://acm.timus.ru/>

Juez en línea disponible las 24 horas, alojado por la Universidad Estatal de los Urales de Rusia. Este sitio contiene problemas de numerosos concursos rusos. Puede ser muy útil para que los estudiantes practiquen. En este sitio se pueden comprender las diferencias entre los problemas rusos y los de otros idiomas.

<http://plg.uwaterloo.ca/~acm00/>

Esta es la página de concursos de la Universidad de Waterloo. Esta universidad lidera actualmente el panorama mundial de los concursos de programación. Organizan concursos casi cada dos semestres y los problemas que se resuelven en ellos sirven de referencia para muchas otras universidades.

<http://oldweb.uwp.edu/academic/mathematics/usaco/> y <http://ace.delos.com/>

Sitio web de formación online para concursos que guía a los principiantes con soluciones paso a paso a los problemas.

<http://www.acmsolver.org/>

Otro sitio de entrenamiento para un concurso de programación en línea de 24 horas desarrollado por "Ahmed Shamsul Arefin", principalmente para programadores novatos, ahora vinculado con los principales jueces en línea del mundo.

<http://www.acm.inf.ethz.ch/ProblemSetArchive.html>

Este sitio es el mejor recurso para los problemas de las finales regionales y mundiales de la ACM. También incluye soluciones y datos de los jueces de muchos concursos, lo que nos será útil para practicar.

<http://acm.uva.es/problemset/replies.html>

Esta página contiene información sobre el significado de las distintas respuestas de los jueces. También es una página de enlace al sitio web acm.uva.es.

<http://contest.uvarov.ru/>

Esta es la página principal del POTM. El concurso POTM significa Concurso del Programador del Mes. El conjunto de problemas de este concurso es diferente al de los concursos de programación convencionales. El organizador da mayor importancia a la optimización que al número de soluciones.

<http://www.acmbeginner.tk>

¿Te resulta muy difícil demostrar que eres un buen programador? Para resolver estos problemas, las dificultades que puedas encontrar en el sistema de evaluación en línea 24 horas y la ayuda y las guías para superarlas se integran en este sitio web "ACMBEGINNER" de MH Rasel.

<http://groups.yahoo.com/group/icpc-l> **http://groups.yahoo.com/group/acm_solver**

Se habla de algoritmos, estructuras de datos y conceptos matemáticos. El grupo está dirigido a equipos, exequipos y cualquier persona interesada en la programación online. Entrenadores y profesores son bienvenidos.

¡Sitio de pioneros!

Consejos para recién llegados, por Shahriar Manzoor

Uno de los mejores programadores y creadores de problemas de acm.uva.es, así como un reconocido creador de problemas de Bangladesh, aconseja a los recién llegados. Actualmente es juez del sitio UVa OJ y WF. También es el creador de problemas del panel de élite del sitio acm.uva.es. Puedes visitar su publicación usando la URL <http://www.acm.org/crossroads/xrds7-5/>

El mundo de Steven, por Steven Halim

Steven Halim publicó su sitio web para quienes buscan resolver problemas, incluyendo tutoriales con trucos y soluciones a problemas, clasificados por nivel de dificultad. En su sitio también ha categorizado los problemas. Este sitio contiene casi todas las pistas de los problemas que ha resuelto. Puedes visitar el sitio web a través del enlace: <http://www.comp.nus.edu.sg/~stevenha/programming/acmoj.html>

Reuber Guerra Duarte

Reuber Guerra Duarte desarrolló este sitio con algunas soluciones (el código fuente real) para problemas que ya había resuelto. Por favor, no envíe directamente sus soluciones para que las juzguen. <http://www.dcc.ufmg.br/~reuber/solutions/index.html>

Archivo de problemas del concurso de programación de Ed, por Ed-Karrels

Ed-karrel, uno de los mejores creadores de problemas, publica este sitio con las soluciones a muchos problemas. Visita el sitio con la URL http://www.karrels.org/Ed/ACM/prob_index.html

Mark Dettinger

Una leyenda viva de los concursos de programación. Fue el principal arquitecto de concursos de la Universidad ULM, Alemania. Ahora esta responsabilidad recae en otra persona: Water Guttman. Si visitas el sitio, encontrarás muchos enlaces útiles, así como muchas anécdotas interesantes de su vida, y te darás cuenta una vez más de que un chico que saca once en matemáticas de cien también puede ser un genio. URL <http://www.informatik.uniulm.de/pm/mitarbeiter/mark/>

Libros**Introducción al algoritmo (CLR)**

Introducción a los algoritmos (Serie de Ingeniería Eléctrica e Informática del MIT) por Thomas H. Cormen, Charles E. Leiserson y Ronald L. Rivest. Ya han publicado la segunda edición, con Stein como cuarto autor. Este libro es imprescindible para ser un buen programador.

Manual de diseño de algoritmos

El Manual de Diseño de Algoritmos de Steven S. Skiena (+ sitio web del libro). Imprescindible para todo buen programador.

El arte de la programación informática

El arte de la programación informática, volúmenes 1-3 (edición en caja) de Donald E. Knuth. Difícil de entender para principiantes, pero vale la pena leerlo.

Matemáticas concretas

Matemáticas Concretas, Segunda Edición + por Ronald L. Graham, Donald E. Knuth, Oren Patashnik. Este libro contiene problemas matemáticos y sus soluciones, pero es muy difícil de entender. A pesar de su dificultad, resulta muy útil para concursos de programación. Se analizan diversas técnicas de formulación.

Teoría de números

El libro "Teoría de los Números" de S.G. Telang contiene mucha información sobre teoría de números. Es recomendable leerlo antes de leer "Matemáticas Concretas". Es fácil de entender.

Matemáticas discretas y su aplicación

El libro "Matemáticas Discretas y sus Aplicaciones" de Kenneth H. Rosen ofrece mucha ayuda práctica sobre matrices de enteros, razonamiento matemático, conteo y problemas de árboles gráficos.

Estructuras de datos usando C y C++

Estructuras de datos con C y C++ por Yedidyah Langsam, Moshe J. Augenstein y Aaron M. Tenenbaum. Segunda edición. Este libro contiene problemas básicos relacionados con estructuras de datos con programación en C.

Desafíos de programación

Desafíos de Programación, por Steven Skiena y Miguel Revilla. No hace falta decir nada sobre estas dos leyendas. Todos los programadores conocen a estas dos leyendas indiscutibles de los concursos de programación. Usa este libro para aprender qué hacer en un concurso de programación.

Concurso de Arte de Programación

Bueno, el concurso "El arte de la programación" es este, organizado por Ahmed Shamsul Arefin.

Guías de lenguajes de programación

<http://www.sgi.com/tech/stl/>

En este sitio encontrarás la documentación de la STL. La STL es fundamental para la programación. Es eficiente y simplifica y organiza el código. Para usarla, es necesario tener conocimientos de plantillas de C++. Al principio, un principiante puede pasar por alto la STL, pero la necesitará más adelante.

www.cplusplus.com / www.cprogramming.com

Excelentes sitios web para programadores de C/C++. Estos sitios contienen documentación, referencias, código fuente y un foro electrónico donde puedes consultar con programadores expertos.

Sitio de matemáticas

mathworld.wolfram.com

MathWorld es una enciclopedia matemática completa e interactiva dirigida a estudiantes, educadores, aficionados a las matemáticas e investigadores. Al igual que la dinámica y siempre cambiante disciplina de las matemáticas, este sitio se actualiza continuamente para incluir nuevo material e incorporar nuevos descubrimientos.

<http://mathforum.org/>

El Foro de Matemáticas es un centro líder en matemáticas y educación matemática en Internet. Su misión es proporcionar recursos, materiales, actividades, interacciones personales y productos y servicios educativos que enriquezcan y apoyen la enseñanza y el aprendizaje en un mundo cada vez más tecnológico.

<http://www.math.com/>

Math.com se dedica a brindar métodos revolucionarios para que estudiantes, padres, maestros y cualquier persona aprenda matemáticas. Combinando principios pedagógicos sólidos con tecnología propia, Math.com ofrece una experiencia única que guía rápidamente al usuario hacia las soluciones que necesita y los productos que desea.

<http://www.math-net.org/>

Math-Net pretende coordinar las actividades de información y comunicación electrónica de la comunidad matemática mundial con el objetivo de mejorar el libre flujo de información dentro de la comunidad.

REFERENCIAS

- [1] "CÓMO MEJORAR EN 24 HORAS PARA JUECES EN LÍNEA", Anupam Bhattacharjee, , Publicado por ACMSolver.org, 2003.
- [2] "World of Seven", MÉTODOS PARA RESOLVER PROBLEMAS DE JUEZ EN LÍNEA DE VALLADOLID, Steven Halim, Universidad Nacional de Singapur, <http://www.comp.nus.edu.sg/~stevenha/>
- [3] ACMSolver.org, sitio web tutorial del concurso de programación ACM/ICPC para Valladolid OJ por Ahmed Shamsul Arefin <http://www.acmsolver.org>
- [4] "Hoja de trucos de informática teórica", Steve Seiden, <http://www.csc.lsu.edu/~seiden>
- [5] "Aprenda C en 21 días", Peter Aitken, Bradley L. Jones, sitio web: <http://www.mcp.com/info/0-672/0-672-31069-4/>
- [6] "Errores comunes en concursos en línea y en tiempo real", Shahriar Manzoor, ACMCross Roads Student Magazine, www.acm.org/crossroads/xrds7-5/contests.html
- [7] Verhoeff, T. Directrices para la elaboración de un conjunto de problemas para un concurso de programación: <http://www.wpa.win.tue.nl/wstomv/publications/guidelines.html>
- [8] Ernst, F., J. Moelands y S. Pieterse. Trabajo en equipo en concursos de programación: 3 * 1 = 4, Cruce de caminos, 3.2. <http://www.acm.org/crossroads/xrds3-2/progcon.html>
- [9] STL (Standard Template Library), Silicon Graphics, Inc., <http://www.sgi.com/tech/stl/>
- [10] ACMBeginner.tk, Herramientas, consejos y tutorial del juez en línea (OJ) de ACM Valladolid por MH Rasel, <http://www.acmbeginner.tk/>
- [11] Brian W. Kernighan- "Programación en C: un tutorial" <http://www.lysator.liu.se/c/bwk-tutor.html>
- [12] Rob Kolstad, Portal de capacitación de USACO, <http://ace.delos.com/>

ÍNDICE

Problemas ad hoc, 16
 Lista de adyacencia, 139
 Matriz de adyacencia, 139
Algoritmo, 19
 Matriz, 74
 Conversión de números de base, 91
 Algoritmo de Bellman-Ford, 164
 Entero grande, 92
 Big Mod, 91, 195
 Búsqueda binaria, 113
 Árbol de búsqueda binaria, 113
 Búsqueda en amplitud, 144 Método de fuerza bruta, 85 Ordenación de burbuja, 107
 Número de Carmichael, 93
 Manejo de colisiones, 115
 combinaciones, 202
 ERROR DE COMPILACIÓN (CE), 15
 campos conectados, 158
Conectividad, 141
 Envoltente convexa, 173
 Contando el cambio, 131
 Combinaciones de conteo, 94
 Clasificación de conteo, 111
ESTRUCTURAS DE DATOS, 72
 depuración, 15
 Decimal a romano, 198
 Descomposición, 89
 Búsqueda en profundidad con profundización iterativa, 150
 Búsqueda en profundidad, 147
 Diccionario, 114
 Algoritmo de Dijkstra, 163
 Grafo acíclico dirigido, 165
 Grafo dirigido, 137
 Divide y vencerás, 88 Divisores y divisibilidad, 95 PROGRAMACIÓN DINÁMICA, 121 Sistema de coordenadas terrestres, 172
 Lista de bordes, 138
 Editar distancia, 127
 Algoritmo de Euclides, 190
 Ciclo de Euler, 170
 Camino de Euler, 170
 Exponenciación, 96
 Factorial, 97
 Algoritmo de Fermat, 104
 Prueba de Fermat: 104
 Fibonacci, 97
 Floyd Warshall, 165
 MCD, 191
 Transposición de grafos, 170
GRÁFICOS, 134
 Máximo Común Divisor (MCD), 99
ALGORITMOS CODICIOSOS, 117

Variaciones de tabla hash, 116
 Conversión de infijo a postfijo, 100
 Búsqueda informada, 154
 Número entero en cualquier base, 197 Problema de Josefo, 200
Jueces, 26
 Algoritmo de Kruskal, 161
MCM, 191
LCS, 205
 Ordenación en tiempo lineal, 111
 Lista enlazada, 76
 Subsecuencia común más larga, 125
 Subsecuencia ascendente/descendente más larga (LIS/LDS), 128
 Mínimo común múltiplo (MCM), 99 Multiplicación de cadena de matrices, 123 Problema de multiplicación de cadena de matrices, 123 Suma máxima de intervalo, 132
MCM, 209
 Distancia MiniMax, 167
 Árboles de expansión mínima, 160
 Árboles de expansión mínima, 159
 Programas de entrada múltiple, 83
 Conversiones numéricas, 196
 Subestructura óptima, 124
 Optimización, 89
 orden de crecimiento, 22
 Otros algoritmos de programación dinámica, 133 PC2, 235
Calculadora de sufijos, 100 pre-cálculo, 89
 Factores primos, 101
 Números primos, 102
 Pruebas de primera calidad, 102
 El algoritmo de Prim, 162
Cola, 79
 Clasificación rápida, 108
 Ordenación por radix, 111
Recursión, 86
BUSCANDO, 113
 Tamiz de Eratóstenes, 105
CLASIFICACIÓN, 106
 algoritmos de ordenación, 106 Pila, 78
STL, 231
 Componentes fuertemente conectados, 171 subgrafos, 142
Simetrías, 90
 Ordenación topológica, 119, 171
 Envoltente transitiva, 166
 Búsqueda no informada, 143
Valladolid OJ, 16
 Variaciones sobre árboles binarios, 114
 Problema de la mochila cero-uno, 130